

Why Networking is Important in Kubernetes → Self-managing

- ↳ Containers are dynamic and short-lived
- ↳ ^{→ Small deployment unit} Pods can be created and destroyed anytime
- ↳ Application must communicate reliably,
- ↳ Provides a cluster-wide networking model.

Kubernetes Networking Model:-

- ↳ Each Pod gets a unique IP address
- ↳ Pods communicate directly without NAT
- ↳ Nodes can communicate with Pods
- ↳ Containers in the same pod share same network.
- ↳ ^{→ Internal communication} What is CNI Container Network Interface?

↳ CNI is a standard used to configure network for containers.

↳ It assigns IP addresses to Pods

↳ It connects Pods to the cluster network.

↳ Ex:- Calico, Flannel, Weave.

Kubernetes Services

↳ A Service provides a stable network endpoint for Pods

↳ Pods are temporary and their IP change

↳ Service provides a fixed IP and DNS name

↳ They load balance traffic across multiple Pods

Type of kibernetics services:-

- ↳ cluster IP - Internal Communication within cluster
- ↳ Nodeport - Exposes Services on node IP and port
- ↳ LoadBalancer - cloud Provider external load Balancer
- ↳ ExternalName - Map Services to external DNS.

Role of kube-proxy:-

- ↳ Runs on every node
 - ↳ watches services and endpoints
 - ↳ create iptables or ipvs rules
 - ↳ Implement services virtual IP routing
- Commands

↳ Kubectl version

- 2) kubectl get nodes
- 3) kubectl describe node _____ ⇒ metadata of node
↓
node-name
- A) kubectl run Sietpod --image = httpd
 ↓ ↓
 Pod name Image name

- 5) kubectl get Pods

- 6) kubectrl describe Pod Siót pod
↓ pod name

10.244.1.21

- #> kubectl exec -it sipdpod -- /bin/bash
 ↓
 pod name

- 8) Kubectl logs gretpod
 ↘ Pod name

- 9) kubectl delete Pod Spetpod
↳ pod name

- 10) kubectl get Pods

11) kubectl create deployment ^{> scaling, self healing} stetdeploy --image=httpd:

12) kubectl get deployment latest

13) kubectl get pods

14) kubectl delete pod (pod name) } > using deployment it cannot delete pod because next pod are being created

15) kubectl get pods

16) kubectl scale deployment stetdeploy --replicas=5

17) kubectl get pods > 5 new pods are created

18) kubectl create deployment test --image=nginx --deployment test
deployment name

19) kubectl get deployment

20) kubectl get pods

⑩ Scale down means replica la count mathura pod hum

21) kubectl delete deployment stetdeploy-test

22) kubectl get deployment

23) kubectl get pods

24) kubectl set image deployment stetdeploy httpd=httpd:tarxio

rolling update

25) kubectl get pods

26) kubectl rollout undo deployment testdeploy

27) kubectl get pods

28)

29) kubectl rollout status deployment testdeploy

- 30) kubectl rollout undo deployment testdeploy
- 31) kubectl rollout status deployment testdeploy
- 32) kubectl get svc
- 33) kubectl delete svc testdeploy
- 34) kubectl get svc
- 35) kubectl expose deployment/testdeploy --type=NodePort
 --port=80

apHaj capital
 ↑ ↑
 NodePort
- 36) kubectl get Services → Overall cluster
- 37) kubectl get Pods
- 38) kubectl get deployment -o wide → Extra metadata

declarative → (resource objects - Yaml files)

Imperative → (Kubernetes CLI)

Namespace → different environment Creating

Understanding Kubernetes objects:-

Kubernetes objects are Persistent entities in the Kubernetes System. Kubernetes uses the entities to represent the state of your cluster: they can describe

↳ what containerized application are running

↳ The resources available to those application

↳ The Policies around how those applications behave
 Such as restart Policies, upgrading, fault tolerance.

deployment resource object

apiVersion: apps/v1

kind: Deployment

metadata:

name: nginx-deployment

Spec:

Selector:

matchLabels:

app: nginx

replicas: 2

template:

metadata:

labels:

app: nginx

Spec:

Containers:

- name: nginx

Image: nginx:1.14.2

Ports:

- containerPort: 80

Pod resource object:-

apiVersion: v1

kind: Pod

metadata:

name: nginx

Spec:

Container:

- name: nginx

Image: nginx:1.14.2

Defining a Service

A Service is an object (the same way that a pod or a ConfigMap is an object). you can create, view or modify service description using kubernetes API.

apiVersion: v1

kind: Service

metadata:

name: my-service

spec:

selector:

app: kubernetes.io/name: MyAPP

ports:

protocol: TCP

port: 80

targetPort: 9376